

Document de Conceptualisation AAG Gomécano CV Matching

Projet : Assistant d'Acquisition Gomécanicien
Type : Proof of Concept (POC)
Contexte : Automatiser le matching entre des candidats Gomécaniciens et des missions opérationnelles à partir de CV PDF bruts. Démonstration via interface web Streamlit.

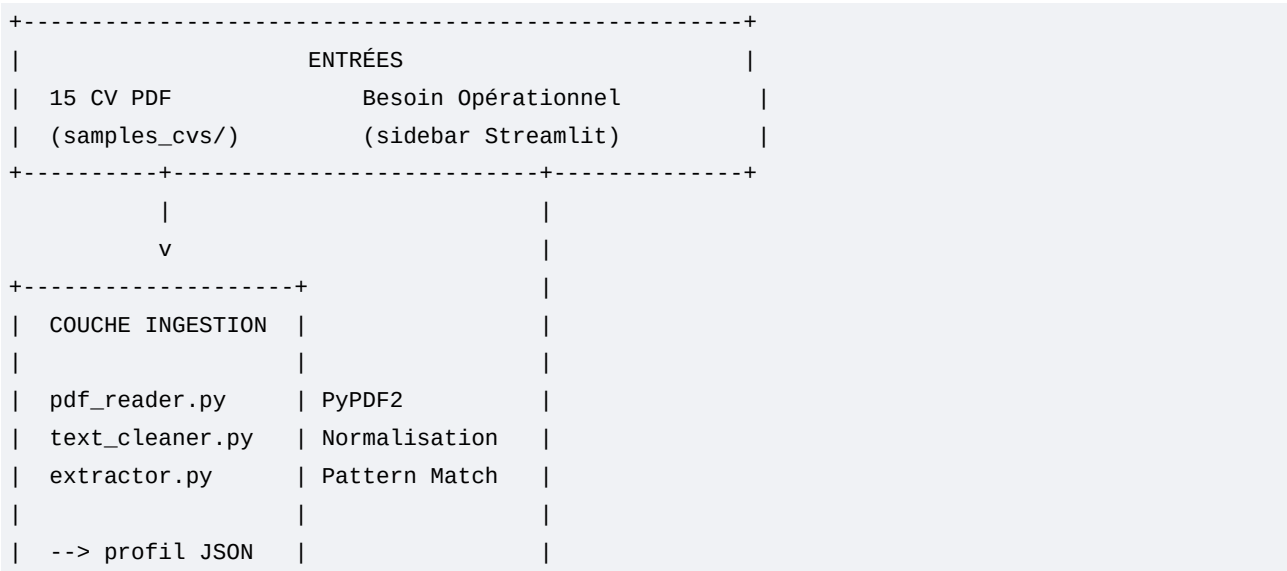
1. Problème Métier

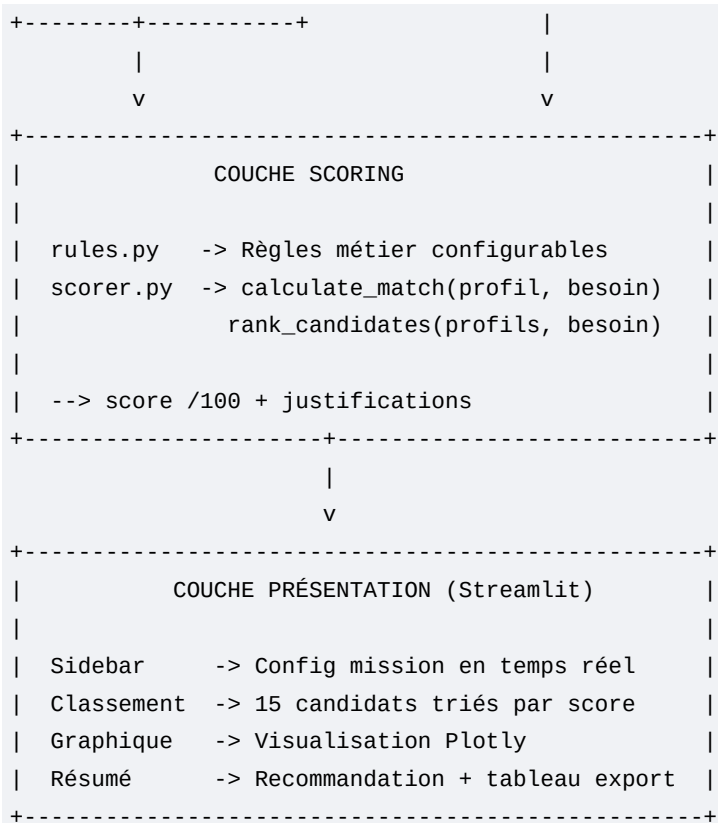
Gomécano recrute des mécaniciens mobiles (Gomécaniciens) pour des missions B2B et B2C.
Le processus de sélection manuel pose trois problèmes :

Problème	Impact
Les CV arrivent en PDF non structurés	Temps de lecture élevé par recruteur
Pas de critères de scoring standardisés	Décisions subjectives, non reproductibles
Volume croissant de candidatures	Goulot d'étranglement humain

Objectif du POC : Valider qu'il est possible d'automatiser l'extraction d'informations clés depuis un CV PDF et de produire un score de compatibilité objectif avec une mission donnée, consultable via un dashboard interactif.

2. Vue d'Ensemble de l'Architecture





3. Pipeline de Données Détaillé

Etape 1 Ingestion (PDF -> Texte)

```

15 CV PDF (samples_cvs/)
|
v
pdf_reader.py
  PyPDF2.PdfReader() -> itère sur chaque page -> concatène le texte brut
|
v
text_cleaner.py
  .lower()           -> normalise la casse
  replace('\n', ' ') -> aplatit le texte
  re.sub(r'\s+', ' ') -> supprime les espaces multiples

```

Etape 2 Extraction de Features (Texte -> JSON structuré)

```

Texte nettoyé
|
+-- extract_name()
|   Heuristique : première ligne avec 2-4 mots en majuscules
|

```

```

+-- extract_city()
|     Lookup dans VILLES_CIBLES (liste PACA + grandes villes FR)
|
+-- extract_skills()
|     Matching par catégorie dans SKILLS_DB :
|     { "electrique": ["batterie", "alternateur", "diagnostic", ...] }
|
`-- extract_experience()
    Regex multi-patterns : "X ans d'expérience", "depuis X ans"
    Sanity check : years < 50

```

Sortie JSON par candidat (exemple) :

```

{
  "nom": "Sophie Martin",
  "ville": "Marseille",
  "competences": ["electrique", "climatisation", "vul"],
  "experience_annees": 8,
  "fichier_source": "CV_Technicienne_VUL.pdf"
}

```

Etape 3 Scoring (Profil + Besoin -> Score)

Score = Critère Géographique + Critère Compétence + Critère Expérience + Bonus

Critère Géographique (poids défaut : 50)

```

Ville exacte match      -> +50 pts
Ville PACA prioritaire -> +25 pts
Hors zone               -> +0 pts

```

Critère Compétence (poids défaut : 30)

```

Compétence requise présente -> +30 pts
Autres compétences          -> +9 pts
Aucune compétence           -> +0 pts

```

Critère Expérience (poids défaut : 20)

```

experience >= minimum -> +20 pts
experience partielle  -> +(20 * exp/min) pts
Non renseignée       -> +0 pts

```

Bonus stratégiques (rules.py)

```

VUL          -> +10 pts
Electrique   -> +5 pts
Climatisation -> +3 pts

```

Etape 4 Affichage Streamlit

```

15 profils JSON (samples_json/)
|

```

```

v
app_streamlit.py
    load_profiles()      -> charge tous les JSON
    rank_candidates()    -> calcule et trie les scores
    Sidebar              -> besoin configurable en temps réel
    Classement           -> affichage avec badges RECOMMANDE / A CONSIDERER
    Graphique Plotly     -> barres horizontales colorées par seuil
    Résumé               -> KPIs + recommandation principale + tableau

```

4. Les 15 Profils de Démonstration

Nom	Ville	Compétences	Exp.
Sophie Martin	Marseille	electrique, climatisa..	8 ans
Clara Moretti	Marseille	electrique, climatisa..	5 ans
Lucie Fontaine	Marseille	electrique, moteur	3 ans
Thomas Bernard	Marseille	moteur, freinage, ele..	0 ans
Olivier Petit	Marseille	moteur, freinage, ele..	7 ans
Clara Moretti	Marseille	electrique, vul, clim..	5 ans
Yasmine Haddad	Aubagne	freinage, electrique,..	4 ans
Antoine Roche	Vitrolles	moteur, electrique	2 ans
Karim Benzarti	Aix-En-Provence	moteur, freinage, ele..	6 ans
Julien Moreau	Toulouse	moteur, freinage, ele..	5 ans
Hugo Blanchard	Lyon	freinage, pneus, carr..	3 ans
Marc Durand	Lyon	moteur, freinage, ele..	12 ans
Jean-Paul Garnier	Bordeaux	moteur, freinage, vul	9 ans
Pierre Lefebvre	Nice	toutes compétences	15 ans
Nathalie Dubois	Paris	carrosserie, pneus	10 ans
Fatima El Amrani	Toulon	electrique	1 an

5. Choix Techniques et Justifications

Choix	Justification	Alternative en production
PyPDF2	Léger, sans dépendances lourde..	pdfplumber, OCR Tesseract pou..
Regex + keyword matching	Zéro coût, 100% déterministe,..	LLM pour extraction sémantique
Fichiers JSON	Pas de setup, lisible, idéal ..	PostgreSQL + ORM SQLAlchemy
Streamlit	Dashboard interactif en quelq..	API FastAPI + frontend dédié

Poids dans sidebar Streamlit	Reconfigurables live sans tou..	Modèle ML de ranking supervisé
Pas de LLM	Valider l'approche sans coût ..	Brancher Claude/GPT sur extra..

6. Limites Connues du POC

Limite	Cause	Impact
CVs scannés non lisibles	PyPDF2 extrait uniquement le ..	Extraction vide
Extraction fragile du nom	Heuristique sur la mise en fo..	Faux négatifs sur CVs non sta..
"diagnostic" détecte électrique	Mot-clé dans SKILLS_DB trop l..	Compétence électrique sur-dét..
Score peut dépasser 100	Bonus non capés	Affichage à normaliser en prod

7. Lien avec la Data et l'IA

Ce POC est un système expert à règles (non probabiliste), pas encore un système IA au sens strict. Il constitue la fondation architecturale d'un futur système IA :

Aujourd'hui (POC)	Demain (Prod IA)
PyPDF2 + Regex	LLM pour extraction sémantique multiformat
Règles métier manuelles (rules.py)	Modèle ML supervisé sur historique missions
Listes de mots-clés (SKILLS_DB)	Embeddings vectoriels + similarité cosinus
Fichiers JSON	Base vectorielle (pgvector, Pinecone)
Poids configurés manuellement	Apprentissage automatique des poids optimaux

Ce POC produit des données d'entraînement potentielles : chaque score calculé + justifications + résultat réel de la mission = un exemple labélisé pour entraîner un futur modèle.

8. Roadmap vers la Production

Phase 1 POC (état actuel)

- Pipeline PDF -> JSON fonctionnel
- Scoring rule-based sur 3 critères
- Dashboard Streamlit avec 15 profils de démo

Phase 2 Robustesse

- Extraction par LLM pour CVs non standardisés

- OCR pour CVs scannés
- Tests unitaires complets
- Validation sur corpus réel 50+ CV

Phase 3 Intégration

- API REST FastAPI exposant `calculate_match()` et `rank_candidates()`
- Base PostgreSQL pour stocker profils et missions
- Intégration CRM Gomécano

Phase 4 IA Supervisée

- Collecte résultats réels (candidat retenu / non retenu)
- Entraînement modèle de ranking sur historique
- Remplacement progressif des règles manuelles
- Monitoring des prédictions en production

9. Structure du Projet

```
aag-gomecano-cv-matching/
|-- data/
|   |-- samples_cvs/      # 15 CV PDF sources
|   |-- samples_json/    # 15 profils JSON (générés par run_ingestion.py)
|   |-- besoin.json      # Besoin par défaut
|-- docs/                # Documentation PDF
|-- scripts/
|   |-- run_ingestion.py  # PDF -> JSON
|   |-- run_demo.py       # Matching CLI (hors demo)
|   |-- generate_sample_cvs.py # Génération des 15 CV
|   |-- md_to_pdf.py      # Génération documentation
|-- src/
|   |-- aag/
|       |-- ingestion/    # pdf_reader, text_cleaner, extractor
|       |-- scoring/      # rules, scorer
|       |-- utils/        # io, logger
|       |-- app_streamlit.py
|-- tests/
```